

CO5151 — Advanced Agentic Artificial Intelligence

Group Project Guidelines Research Track and Application Track

Faculty of Computer Science and Engineering, HCMUT
Lecturer: Lê Xuân Bách

Semester HK261 — version of September 2026

The project is worth **50% of the final course grade**. Groups of **3–4 students** design, implement, and evaluate one complete agentic AI system: proposed in Week 2, checkpointed in Week 7, and defended over two rounds in Weeks 12–13. Course website: <https://lexuanbach.github.io/5151>

Contents

1	Overview and the two tracks	1
1.1	Choosing a track	2
2	Topic-quality gate	2
3	Technical requirements (both tracks)	3
3.1	Track-specific additions	3
4	Timeline and deliverables	4
5	Rubrics	5
5.1	Common criteria and weights	5
5.2	Research Track descriptors	5
5.3	Application Track descriptors	5
5.4	Individual adjustment	5
6	Report templates	5
6.1	Proposal — D1, 3–4 pages, both tracks	5
6.2	Progress report — D2, 4–6 pages, both tracks	6
6.3	Final report — D3, 6–8 pages, conference style	6
7	Suggested themes, mapped to tracks	6
8	Compute, API, and integrity policy	7

1 Overview and the two tracks

Every group builds one agentic AI system and must satisfy the same technical core (the topic-quality gate of Section 2 and requirements R1–R9 of Section 3). What differs between the tracks is the *primary claim* your project makes and how that claim is judged:

- **Research Track** — your project **tests a falsifiable hypothesis** about agentic AI and produces a credible finding or a reusable research artifact (a benchmark, a dataset, a technique, a measurement). The system is the *instrument*; the finding is the product. A well-supported negative result is a fully acceptable outcome.
- **Application Track** — your project **solves an actual problem for identified users**. The working system is the product; the claim is that it delivers real value robustly, safely, and at a defensible cost. “It works in a happy-path demo” is not the bar — “it survives realistic use and we measured how well” is.

1.1 Choosing a track

Declare your track in the D1 proposal. Use the comparison below; when in doubt, ask yourself which failure would embarrass you more — an unconvincing experiment (research) or a fragile system (application).

	Research Track	Application Track
Primary output	A finding or reusable artifact	A working system with users in mind
Core question	“Is this hypothesis true, under what conditions?”	“Does this system solve the problem well enough, at what cost?”
Related work	Deep: position against 10–15 papers; state the gap precisely	Focused: 5–10 papers/systems; explain why existing tools fall short
Evaluation center of gravity	Controlled comparison: baselines, ablations, statistics	End-to-end task success, robustness, cost/latency at realistic load
Demo role	Illustrates the experimental setup and a representative trace	Central: the system running live on realistic inputs, failure handling included
Report style	Conference paper (claim–evidence)	System/experience report (problem–design–validation)
Typical risk	Result is noise: too few seeds, weak baseline, contaminated benchmark	Breadth without depth: many features, none robust, no honest measurement

Table 1: The two tracks at a glance. Both tracks satisfy the same R1–R9 core.

Both tracks are graded to the same standard and the same weightings (Section 5); neither is the “easy” option. The track changes what the rubric’s criteria *look for*, not how much they count.

2 Topic-quality gate

Checked at D1, *before* R1–R9 apply. A proposal that satisfies every requirement on paper but fails one of these still comes back “changes required.”

Novel — not a reproduction.

Not a direct copy of a tutorial, course example, or published demo. At least one element — the task, the combination of advanced axes, the evaluation, or the deployment context — is your own choice. *Research*: the hypothesis is not already settled by the papers you cite. *Application*: the problem–solution pairing is not already served by an existing tool you could simply configure.

Non-trivial — cannot be vibe-coded in a day.

Requires genuine design decisions: real integration across ≥ 3 advanced axes, real failure handling, and an evaluation that surfaces a non-obvious trade-off.

Meaningful — has real value.

Research: the finding would change what a practitioner or researcher does next. *Application*: identifiable users exist and the problem costs them real time, money, or risk today.

Feasible — fits ~10 weeks.

Achievable by a team of 3–4 between the D1 proposal (Week 2) and Round 1 (Week 12), given your stated skills and your compute and API budget. Your work plan must show this, including a fallback scope.

3 Technical requirements (both tracks)

#	Checked at	Requirement
R1	D1, D2, D3	Clear agent architecture. Reasoning–acting loop and/or planning and/or multi-agent orchestration, documented as a diagram plus a state/control-flow description.
R2	D2, D3	Tools / MCP integration. At least one MCP server (your own or third-party) or an equivalent function-calling toolset. Each tool’s permission level is documented as read-only, reversible-write, or irreversible-write.
R3	D2, D3	Memory. An explicit design: short-term/context management plus at least one long-term or episodic store, with a stated write policy and retrieval policy.
R4	D1, D3	≥ 3 advanced axes beyond tools and memory: planning/search, reflection/verification, multi-agent, RAG, RL fine-tuning, computer use, or guardrails. Depth beats breadth.
R5	D2, D3	Quantitative evaluation. A benchmark or curated task set (≥ 20 tasks), a success metric with cost and latency reported alongside, ≥ 2 system configurations (an ablation), ≥ 3 runs/seeds with variance, and an error analysis categorizing why failures happened.
R6	D1, D3	Security & safety analysis. A threat model (assets, entry points, attacker capabilities); tests for direct and indirect prompt injection (≥ 10 attack cases); permission control / least privilege on tools; monitoring and audit logging.
R7	W7, W12, W13	Demo. A live demo backed by a recorded video — network or API outages must not stop your defense.
R8	D4	Reproducible code. README, pinned dependencies, <code>.env.example</code> , a one-command run, evaluation scripts, seeds, traces/logs, and a cost report.
R9	D3	Report. 6–8 pages, conference style, with an AI-use statement and a contribution statement.

Table 2: The nine core requirements. “Checked at” names the deliverable where each is assessed.

3.1 Track-specific additions

Research Track — add:

- **RT1 — Hypothesis.** One falsifiable, scoped hypothesis stated in the proposal (“System/technique X improves metric M over baseline B on task set T under budget C”), plus what evidence would *refute* it.
- **RT2 — Baselines.** At least one credible external baseline compared at matched cost or token budget — not only your own system with a module removed.
- **RT3 — Statistical care.** Means with variance across seeds; do not claim a difference that lies within one or two standard errors. Discuss contamination risk for any public benchmark used.
- **RT4 — Reusable artifact.** The task set, harness, or technique packaged so another group could rerun or extend it.

Application Track — add:

- **AT1 — Users and scenario.** Named user group and ≥ 3 concrete usage scenarios (including one adversarial or careless user), validated with at least an informal walkthrough with a real potential user.
- **AT2 — Robustness.** Explicit handling for tool failure, timeout, malformed model output, and retry safety on non-idempotent tools; demonstrated in the demo, not just claimed.
- **AT3 — Operational envelope.** Measured cost and latency per task at realistic usage, plus a stated scale limit (“works up to N documents / M users because...”).
- **AT4 — Guarded actions.** Every irreversible-write tool sits behind a confirmation, permission check, or sandbox; show the audit trail for one guarded action end-to-end.

4 Timeline and deliverables

All submission deadlines are at 23:59. Class meets Wednesdays 18:00–20:29, room B4-305. Dates follow the current HK261 timetable; later instructor or registrar announcements take priority.

Week	Date	Project activity
W1	Wed 9 Sep	Course opens. Form groups of 3–4; pick a track and a theme. Draft the formative idea 1-pager (problem, users, why an agent, candidate axes).
–	Sun 13 Sep	D1 due: proposal (3–4 pp.).
W2	Wed 16 Sep	5’ proposal pitch per group. Verdict: approved / changes required / resubmit.
W3	Wed 23 Sep	Resubmissions close. A proposal still unapproved in Week 3 loses one point on the architecture criterion. Build starts in earnest.
W4–W6	30 Sep – 21 Oct	Core build: architecture (R1), tools/MCP (R2), memory (R3). Stand up the evaluation harness early — design the evaluation before the system flatters it. <i>No class Wed 14 Oct (midterm week).</i>
–	Sun 25 Oct	D2 due: interim progress report (4–6 pp.).
W7	Wed 28 Oct	Checkpoint: 8’ live demo. First numbers (even on 5–10 tasks), threat model v1 with first injection tests. Status: on track / at risk / re-scope.
W8–W11	4 Nov – 25 Nov	Full evaluation runs (R5), security analysis (R6), hardening. <i>Research:</i> baselines, ablations, seeds. <i>Application:</i> robustness, guarded actions, user walkthrough.
–	Sun 29 Nov	D3 v1 + D4 due: final report v1 + code, tagged v1.
W12	Wed 2 Dec	Round 1 defense: 10’ presentation with live demo + 5’ Q&A, reviewed by a peer group and the instructor.
W13	Wed 9 Dec	Round 2 defense: defend what changed since Round 1 and why; final results, limitations, extensibility.
–	Fri 11 Dec	Final submission: D3 v2, contribution statements, intra-group peer evaluation, and D4 tagged final .

Table 3: Week-by-week timeline. Seminar 1 runs W3–W6 and Seminar 2 runs W8–W11 in parallel with the project.

Weighting inside the project component (PRJ = 50% of the course). Proposal (D1) **20%** · interim progress report (D2) **30%** · final report + code + presentation (D3/D4, both defense rounds) **50%**. Within that final 50%: report 30/50, reproducible code 10/50, presentation and individual contribution 10/50 (Round 1 counts 60% and Round 2 counts 40% of the presentation share). The milestone weights say *when* marks are earned; the rubric in Section 5 says *how* the work is judged.

Late policy. –10% per day for up to 3 days; after that, the submission receives 0.

5 Rubrics

Both tracks use the same five criteria and weights. The descriptors below state what earns a high score in each track.

5.1 Common criteria and weights

Criterion	Weight
C1 — Architecture design & technical difficulty	25%
C2 — Implementation & demo	25%
C3 — Quantitative evaluation & security analysis	25%
C4 — Report & reproducibility	15%
C5 — Presentation, defense & individual contribution	10%

5.2 Research Track descriptors

What earns a high score	
C1 (25%)	Architecture is motivated <i>by the hypothesis</i> : every component either serves the experiment or is explicitly out of scope. ≥ 3 axes integrated coherently; design trade-offs argued against at least one alternative the group considered and rejected.
C2 (25%)	The experimental instrument runs end-to-end and is trustworthy: deterministic where it should be, logged everywhere, demo shows a representative trace and a failure case. Clean, reviewable code.
C3 (25%)	The finding is credible: external baseline at matched budget (RT2), ablation isolating the claimed mechanism, ≥ 3 seeds with variance (RT3), cost/latency reported, contamination discussed, and an error analysis that explains — not just counts — the failures. Full threat model with ≥ 10 injection cases and findings acted on. A well-argued negative result scores as highly as a positive one.
C4 (15%)	Conference-quality 6–8 pp.; every claim matches the evidence; explicit threats-to-validity section; the artifact (RT4) reruns from a clean checkout with one command.
C5 (10%)	Both talks make the claim–evidence chain audible; Round-1 feedback fully addressed in Round 2; contribution consistent with commits and peer evaluation.

Table 4: Research-track rubric descriptors.

5.3 Application Track descriptors

5.4 Individual adjustment

With documented evidence, the intra-group peer evaluation, commit history, and Q&A performance can apply an individual factor from **0.8 to 1.1** to the group project score (capped at 10/10). Round-1 feedback that is ignored in Round 2 costs points on C5.

6 Report templates

6.1 Proposal — D1, 3–4 pages, both tracks

Topic-quality self-assessment (novel / non-trivial / meaningful / feasible, in your own words) · **declared track** · title, group & roles · problem & users · related systems & papers (research: 10–15 refs; application: 5–10) · proposed architecture (diagram, control loop, tools/MCP with permission levels, memory design, orchestration — mark your ≥ 3 axes) · evaluation plan (task set, metrics, baselines, seeds) · *research only*: the hypothesis (RT1) · *application only*: usage scenarios (AT1) · threat model v0 · work plan to W7 and W12 · risks & fallback scope · AI-use statement.

What earns a high score	
C1 (25%)	Architecture is motivated <i>by the users’ problem</i> : the simplest design that meets the need, with complexity added only where justified (workflows-vs-agents guidance). ≥ 3 axes integrated coherently; permission levels (R2) and guarded actions (AT4) are first-class design elements, not afterthoughts.
C2 (25%)	Robust end-to-end system: live demo on realistic inputs including an induced failure that the system handles gracefully (AT2); retry-safe tool use; clean code; the audit trail for one guarded action shown end-to-end.
C3 (25%)	Honest measurement of value: ≥ 20 -task set drawn from realistic usage, success + cost + latency per task, an ablation showing which component the value depends on, ≥ 3 runs with variance, an operational envelope (AT3), and an error analysis. Full threat model with ≥ 10 injection cases — including at least one through the system’s own data (documents, tickets, retrieved content) — and findings acted on.
C4 (15%)	6–8 pp. system report; claims match evidence; limitations and threats-to-validity stated plainly (including what was <i>not</i> tested); one-command reproduction verified; actual spend reported.
C5 (10%)	Both talks tell the problem–solution–evidence story clearly; Round-1 feedback fully addressed in Round 2; contribution consistent with commits and peer evaluation.

Table 5: Application-track rubric descriptors.

6.2 Progress report — D2, 4–6 pages, both tracks

Delta from proposal (scope changes, with reasons) · implemented architecture (what runs end-to-end today) · tools/MCP and memory: implemented vs. planned · evaluation harness with first numbers and cost so far · threat model v1 with first injection results · plan to W12 · AI-use statement.

6.3 Final report — D3, 6–8 pages, conference style

§	Research Track	Application Track
1	Introduction: gap and hypothesis	Introduction: users, problem, why an agent
2	Background & related work (positioned)	Background & related systems (why they fall short)
3	System design as experimental instrument	System design and rationale (simplicity argued)
4	Implementation	Implementation, robustness & guarded actions
5	Evaluation: setup, baselines, ablations, results, cost, error analysis	Evaluation: task set, success/cost/latency, ablation, operational envelope, error analysis
6	Security & safety analysis	Security & safety analysis
7	Limitations, threats to validity, extensibility	Limitations, threats to validity, deployment path
8	Ethics & responsible use	Ethics & responsible use
9	Conclusion: the finding, and what it changes	Conclusion: the value delivered, and what remains

Table 6: Final-report structure by track. Both end with references and an appendix: reproducibility (commands, seeds, hardware, budget), contribution statement, AI-use statement.

7 Suggested themes, mapped to tracks

Deliberately open — propose your own theme if it passes the gate. Most themes fit either track depending on the claim you make with them.

#	Theme	Fit
1	Research assistant agent: RAG/GraphRAG + cross-session memory + MCP tools (arXiv, Semantic Scholar); injection via retrieved papers	R / A
2	Repository maintenance agent: issue triage → patch → sandboxed tests; evaluated with pass@1 / pass ^k	R / A
3	Data-analysis / BI agent: NL → SQL/pandas via MCP; SQL-level least privilege against injection via table contents	A
4	Multi-agent software team vs. a single agent at matched budget; failure-mode and collusion analysis	R
5	Web/computer-use agent in a bounded domain; safety gates on irreversible actions	A
6	Customer-support agent under policies (τ -bench style) with human escalation; social-engineering defenses	R / A
7	Secure agent gateway: an MCP proxy with permission control or CaMeL-style taint tracking, on AgentDojo’s utility/security trade-off	R
8	Long-horizon personal assistant: episodic/semantic memory design; memory-poisoning attacks and detection	R
9	Agentic RAG for Vietnamese legal/administrative documents; retrieval-decision policy, grounding, hallucination analysis	R / A
10	RL fine-tuning for tool use: GRPO / verifiable rewards on a small open model vs. prompting a larger model at equal cost	R

Table 7: Suggested themes. “Fit” marks the track(s) each theme most naturally supports.

8 Compute, API, and integrity policy

- You may use hosted LLM APIs (course-provided credits if available, or your own keys), local open-weight models via Ollama/vLLM, or university GPU resources where available. The proposal must state the planned model(s) and an estimated budget; the final report must report actual spend.
- Frameworks are your choice — LangGraph, the Anthropic/OpenAI SDKs, AutoGen, CrewAI, smolagents, or plain Python — but the report must explain what the framework does for your architecture and what you built by hand.
- **Non-negotiable:** no evaluation run without cost logging. Use local models during development; reserve API spend for the evaluation runs that go into your report.
- AI assistance is permitted and expected in this course, and it must be declared: every deliverable carries an AI-use statement saying what tools were used, for what, and what you verified yourself. The contribution statement and commit history must reflect who actually did the work.